

# Architecture Justification

**NAME:-**Prasanna Pujari

**SRN:-**PES1UG24CS596

## **Problem Statement #49 — Webhook Ingestion & Retry Mechanism Hub**

### **1. Architectural Choice**

We chose Microservices Architecture for the Webhook Ingestion & Retry Mechanism Hub. In this architecture, each responsibility in the webhook pipeline is implemented as its own service instead of being part of one large application. This fits a system where ingestion, validation, delivery, retries and monitoring do not all need to run or scale together.

### **2. Scenario-Specific Reasons**

**Reason 1:** The system has clearly separated responsibilities: the Webhook Ingestion Service accepts incoming events, the Validation Service checks their structure, the Delivery/Forwarding Service sends them to internal targets, the Retry Manager handles failed attempts, the Dead-Letter Queue stores permanently failed messages, and the Monitoring & Integration Management Service exposes status and configuration. Microservices lets each of these be built, deployed and updated on its own, instead of forcing a change to one part of the pipeline, such as adjusting retry timing, to involve redeploying the whole system.

**Reason 2:** The gateway depends on exponential backoff retries (1s → 2s → 4s → 8s) and a dead-letter queue for messages that keep failing. Because the Retry Manager and Dead-Letter Queue are separate services from the Delivery/Forwarding Service, a slow or failing delivery does not block the ingestion of new webhooks. If the delivery path is experiencing failures, the Retry Manager can keep retrying independently while the Webhook Ingestion Service keeps accepting new events, which supports the system's high reliability target.

### **3. Security Advantage**

One security advantage is that splitting the system into services with distinct interfaces limits how much of the system is reachable from any single entry point. For example, the Webhook Ingestion API is the only interface exposed to the External Webhook Provider, while the Integration Configuration API is reachable only by the Integration Developer and System Operator. This reduces unnecessary access between components and keeps configuration and monitoring functionality separated from the high-volume ingestion path.

### **4. Performance Benefit**

One performance benefit is that services on the high-load path, particularly the Webhook Ingestion Service and the Delivery/Forwarding Service, can be scaled independently of lower-traffic services such as Monitoring & Integration Management. Since the system needs to support up to 2,500 requests per second, this means ingestion or delivery instances can be increased without also scaling components that do not need the same capacity.

### **5. Conclusion**

Overall, Microservices Architecture supports the reliability, retry handling and throughput requirements of the Webhook Ingestion & Retry Mechanism Hub by allowing each responsibility to fail, scale and be maintained independently.